11

Kernels for structured data: strings, trees, etc.

Probably the most important data type after vectors and free text is that of symbol strings of varying lengths. This type of data is commonplace in bioinformatics applications, where it can be used to represent proteins as sequences of amino acids, genomic DNA as sequences of nucleotides, promoters and other structures. Partly for this reason a great deal of research has been devoted to it in the last few years. Many other application domains consider data in the form of sequences so that many of the techniques have a history of development within computer science, as for example in stringology, the study of string algorithms.

Kernels have been developed to compute the inner product between images of strings in high-dimensional feature spaces using dynamic programming techniques. Although sequences can be regarded as a special case of a more general class of structures for which kernels have been designed, we will discuss them separately for most of the chapter in order to emphasise their importance in applications and to aid understanding of the computational methods. In the last part of the chapter, we will show how these concepts and techniques can be extended to cover more general data structures, including trees, arrays, graphs and so on.

Certain kernels for strings based on probabilistic modelling of the data-generating source will not be discussed here, since Chapter 12 is entirely devoted to these kinds of methods. There is, however, some overlap between the structure kernels presented here and those arising from probabilistic modelling covered in Chapter 12. Where appropriate we will point out the connections.

The use of kernels on strings, and more generally on structured objects, makes it possible for kernel methods to operate in a domain that traditionally has belonged to syntactical pattern recognition, in this way providing a bridge between that field and statistical pattern analysis.

11.1 Comparing strings and sequences

In this chapter we consider the problem of embedding two sequences in a high-dimensional space in such a way that their relative distance in that space reflects their similarity and that the inner product between their images can be computed efficiently. The first decision to be made is what similarity notion should be reflected in the embedding, or in other words what features of the sequences are revealing for the task at hand. Are we trying to group sequences by length, composition, or some other properties? What type of patterns are we looking for? This chapter will describe a number of possible embeddings providing a toolkit of techniques that either individually or in combination can be used to meet the needs of many applications.

Example 11.1 The different approaches all reduce to various ways of counting substrings or subsequences that the two strings have in common. This is a meaningful similarity notion in biological applications, since evolutionary proximity is thought to result both in functional similarity and in sequence similarity, measured by the number of insertions, deletions and symbol replacements. Measuring sequence similarity should therefore give a good indication about the functional similarity that bioinformatics researchers would like to capture. ■

We begin by defining what we mean by a string, substring and subsequence of symbols. Note that we will use the term substring to indicate that a string occurs contiguously within a string s, and subsequence to allow the possibility that gaps separate the different characters resulting in a non-contiguous occurrence within s.

Definition 11.2 [Strings and substrings] An *alphabet* is a finite set Σ of $|\Sigma|$ symbols. A *string*

$$s = s_1 \dots s_{|s|},$$

is any finite sequence of symbols from Σ, including the empty sequence denoted ε, the only string of length 0. We denote by Σ^n the set of all finite strings of length n, and by Σ^* the set of all strings

$$\Sigma^* = \bigcup_{n=0}^{\infty} \Sigma^n.$$

We use $[s = t]$ to denote the function that returns 1 if the strings s and t are identical and 0 otherwise. More generally $[p(s,t)]$ where $p(s,t)$ is a boolean

expression involving s and t, returns 1 if $p(s,t)$ is true and 0 otherwise. For strings s, t, we denote by $|s|$ the *length* of the string s and by st the string obtained by concatenating the strings s and t. The string t is a substring of s if there are (possibly empty) strings u and v such that

$$s = utv.$$

If $u = \varepsilon$, we say that t is a *prefix* of s, while if $v = \varepsilon$, t is known as a *suffix*. For $1 \leq i \leq j \leq |s|$, the string $s(i:j)$ is the substring $s_i \dots s_j$ of s. The substrings of length k are also referred to as *k-grams* or *k-mers*. ■

Definition 11.3 [Subsequence] We say that u is a *subsequence* of a string s, if there exist indices $\mathbf{i} = (i_1, \dots, i_{|u|})$, with $1 \leq i_1 < \cdots < i_{|u|} \leq |s|$, such that $u_j = s_{i_j}$, for $j = 1, \dots, |u|$. We will use the short-hand notation $u = s(\mathbf{i})$ if u is a subsequence of s in the positions given by $\mathbf{i}$. We denote by $|\mathbf{i}| = |u|$ the number of indices in the sequence, while the length $l(\mathbf{i})$ of the subsequence is $i_{|u|} - i_1 + 1$, that is, the number of characters of s covered by the subsequence. The empty tuple is understood to index the empty string ε. Throughout this chapter we will use the convention that bold indices $\mathbf{i}$ and $\mathbf{j}$ range over strictly ordered tuples of indices, that is, over the sets

$$I_k = \{(i_1, \dots, i_k) : 1 \leq i_1 < \cdots < i_k\} \subset \mathbb{N}^k,\ k = 0, 1, 2, \dots.$$

■

Example 11.4 For example, the words in this sentence are all strings from the alphabet $\Sigma = \{\mathtt{a}, \mathtt{b}, ..., \mathtt{z}\}$. Consider the string $s =$"`kernels`". We have $|s| = 7$, while $s(1:3) =$"`ker`" is a prefix of s, $s(4:7) =$"`nels`" is a suffix of s, and together with $s(2:5) =$"`erne`"all three are substrings of s. The string $s(1,2,4,7) =$"`kens`" is a subsequence of length 4, whose length $l(1,2,4,7)$ in s is 7. Another example of a subsequence is $s(2,4,6) =$"`enl`". ■

Remark 11.5 [Binary representation] There is a 1–1 correspondence between the set I_k and the binary strings with exactly k 1s. The tuple $(i_1, \dots, i_k)$ corresponds to the binary string with 1s in positions i_j, $j = 1, \dots, k$. For some of the discussions below it can be easier to think in terms of binary strings or vectors, as in Example 11.18. ■

Remark 11.6 [On strings and sequences] There is some ambiguity between the terms 'string' and 'sequence' across different traditions. They are sometimes used equivalently both in computer science and biology, but in

most of the computer science literature the term 'string' requires contiguity, whereas 'sequence' implies only order. This makes a difference when moving to substrings and subsequences. What in the biological literature are called 'sequences' are usually called 'strings' in the computer science literature. In this chapter we will follow the computer science convention, though frequently including an indication as to whether the given substring is contiguous or not in order to minimise possible confusion. ■

Embedding space All the kernels presented in this chapter can be defined by an explicit embedding map from the space of all finite sequences over an alphabet Σ to a vector space F. The coordinates of F are indexed by a subset I of strings over Σ, that is by a subset of the input space. In some cases I will be the set Σ^p of strings of length p giving a vector space of dimension $|\Sigma|^p$, while in others it will be the infinite-dimensional space indexed by Σ^*. As usual we use ϕ to denote the feature mapping

$$\phi: s \longmapsto (\phi_u(s))_{u \in I} \in F.$$

For each embedding space F, there will be many different maps ϕ to choose from. For example, one possibility is to set the value of the coordinate $\phi_u(s)$ indexed by the string u to be a count of how many times u occurs as a contiguous substring in the input string s, while another possible choice is to count how many times u occurs as a (non-contiguous) subsequence. In this second case the weight of the count can also be tuned to reflect how many gaps there are in the different occurrences. We can also count approximate matches appropriately weighting for the number of mismatches.

In the next section we use the example of a simple string kernel to introduce some of the techniques. This will set the scene for the subsequent sections where we develop string kernels more tuned to particular applications.

11.2 Spectrum kernels

Perhaps the most natural way to compare two strings in many applications is to count how many (contiguous) substrings of length p they have in common. We define the *spectrum of order p* (or *p-spectrum*) of a sequence s to be the histogram of frequencies of all its (contiguous) substrings of length p. Comparing the p-spectra of two strings can give important information about their similarity in applications where contiguity plays an important role. We can define a kernel as the inner product of their p-spectra.

Definition 11.7 [The p-spectrum kernel] The feature space F associated with the p-spectrum kernel is indexed by $I = \Sigma^p$, with the embedding given by

$$\phi_u^p(s) = |\{(v_1, v_2) : s = v_1 u v_2\}|, \ u \in \Sigma^p.$$

The associated kernel is defined as

$$\kappa_p(s,t) = \langle \phi^p(s), \phi^p(t) \rangle = \sum_{u \in \Sigma^p} \phi_u^p(s)\, \phi_u^p(t).$$

■

Example 11.8 [2-spectrum kernel] Consider the strings "bar", "bat", "car" and "cat". Their 2-spectra are given in the following table:

ϕ	ar	at	ba	ca
bar	1	0	1	0
bat	0	1	1	0
car	1	0	0	1
cat	0	1	0	1

with all the other dimensions indexed by other strings of length 2 having value 0, so that the resulting kernel matrix is:

K	bar	bat	car	cat
bar	2	1	1	0
bat	1	2	0	1
car	1	0	2	1
cat	0	1	1	2

■

Example 11.9 [3-spectrum kernel] As a further example, consider the following two sequences

s ="statistics"
t ="computation"

The two strings contain the following substrings of length 3

"sta", "tat", "ati", "tis", "ist", "sti", "tic", "ics"
"com", "omp", "mpu", "put", "uta", "tat", "ati", "tio", "ion"

and they have in common the substrings "tat" and "ati", so their inner product would be $\kappa(s,t) = 2$. ■

Many alternative recursions can be devised for the calculation of this kernel with different costs and levels of complexity. We will present a few of them spread through the rest of this chapter, since they illustrate different points about the design of string kernels.

In this section we present the first having a cost $O(p|s||t|)$. Our aim in later sections will be to show how the cost can be reduced to $O(p(|s|+|t|)) = O(p\max(|s|,|t|))$ making it linear in the length of the longer sequence.

The first method will nonetheless be useful because it will introduce some methods that are important when considering more sophisticated kernels, for example ones that can tolerate partial matches and insertions of irrelevant symbols. It will also illustrate an important consideration that makes it possible to use partial results of the evaluation of one entry in the kernel matrix to speed up the computation of other entries in the same row or column. In such cases the complexity of computing the complete kernel matrix can be less than $O(\ell^2)$ times the cost of evaluating a single entry.

Computation 11.10 [p-spectrum recursion] We can first define an 'auxiliary' kernel known as the k-suffix kernel to assist in the computation of the p-spectrum kernel. The k-suffix kernel $\kappa_k^S(s,t)$ is defined by

$$\kappa_k^S(s,t) = \begin{cases} 1 & \text{if } s = s_1 u,\ t = t_1 u, \text{ for } u \in \Sigma^k \\ 0 & \text{otherwise.} \end{cases}$$

Clearly the evaluation of $\kappa_k^S(s,t)$ requires $O(k)$ comparisons, so that the p-spectrum kernel can be evaluated using the equation

$$\kappa_p(s,t) = \sum_{i=1}^{|s|-p+1} \sum_{j=1}^{|t|-p+1} \kappa_p^S(s(i:i+p), t(j:j+p))$$

in $O(p|s||t|)$ operations. ■

Example 11.11 The evaluation of the 3-spectrum kernel using the above recurrence is illustrated in the following Table 11.1 and 11.2, where each entry computes the kernel between the corresponding prefixes of the two strings. ■

Remark 11.12 [Computational cost] The evaluation of one row of the table for the p-suffix kernel corresponds to performing a search in the string t for the p-suffix of a prefix of s. Fast string matching algorithms such as the Knuth–Morris–Pratt algorithm can identify the matches in time $O(|t|+p) = O(|t|)$, suggesting that the complexity could be improved to

DP : κ_3^S	ε	c	o	m	p	u	t	a	t	i	o	n
ε	0	0	0	0	0	0	0	0	0	0	0	0
s	0	0	0	0	0	0	0	0	0	0	0	0
t	0	0	0	0	0	0	0	0	0	0	0	0
a	0	0	0	0	0	0	0	0	0	0	0	0
t	0	0	0	0	0	0	0	0	1	0	0	0
i	0	0	0	0	0	0	0	0	0	1	0	0
s	0	0	0	0	0	0	0	0	0	0	0	0
t	0	0	0	0	0	0	0	0	0	0	0	0
i	0	0	0	0	0	0	0	0	0	0	0	0
c	0	0	0	0	0	0	0	0	0	0	0	0
s	0	0	0	0	0	0	0	0	0	0	0	0

Table 11.1. *3-spectrum kernel between two strings.*

DP : κ_3	ε	c	o	m	p	u	t	a	t	i	o	n
ε	0	0	0	0	0	0	0	0	0	0	0	0
s	0	0	0	0	0	0	0	0	0	0	0	0
t	0	0	0	0	0	0	0	0	0	0	0	0
a	0	0	0	0	0	0	0	0	0	0	0	0
t	0	0	0	0	0	0	0	0	1	1	1	1
i	0	0	0	0	0	0	0	0	1	2	2	2
s	0	0	0	0	0	0	0	0	1	2	2	2
t	0	0	0	0	0	0	0	0	1	2	2	2
i	0	0	0	0	0	0	0	0	1	2	2	2
c	0	0	0	0	0	0	0	0	1	2	2	2
s	0	0	0	0	0	0	0	0	1	2	2	2

Table 11.2. *3-spectrum kernel between strings.*

$O(|s||t|)$ operations using this approach. We do not pursue this further since we will consider even faster implementations later in this chapter. ■

Remark 11.13 [Blended spectrum kernel] We can also consider a 'blended' version of this kernel $\tilde{\kappa}_p$, where the spectra corresponding to all values $1 \leq d \leq p$ are simultaneously compared, and weighted according to λ^d. This requires the use of an auxiliary p-suffix kernel $\tilde{\kappa}_p^S$ that records the level of similarity of the suffices of the two strings. This kernel is defined recursively as follows

$$\tilde{\kappa}_0^S(sa, tb) = 0,$$

$$\tilde{\kappa}_p^S(sa,tb) = \begin{cases} \lambda^2\left(1+\tilde{\kappa}_{p-1}^S(s,t)\right), & \text{if } a=b; \\ 0, & \text{otherwise.} \end{cases}$$

The blended spectrum kernel can be defined using the same formula as the p-spectrum kernel as follows

$$\tilde{\kappa}_p(s,t) = \sum_{i=1}^{|s|-p+1}\sum_{j=1}^{|t|-p+1} \tilde{\kappa}_p^S\left(s(i:i+p), t(j:j+p)\right),$$

resulting in the same time complexity as the original p-spectrum kernel. ■

As mentioned above later in this chapter we will give a more efficient method for computing the p-spectrum kernel. This will be based on a tree-like data structure known as a trie. We first turn to considering subsequences rather than substrings as features in our next example.

11.3 All-subsequences kernels

In this section we consider a feature mapping defined by all contiguous or non-contiguous subsequences of a string.

Definition 11.14 [All-subsequences kernel] The feature space associated with the embedding of the all-subsequences kernel is indexed by $I = \Sigma^*$, with the embedding given by

$$\phi_u(s) = |\{\mathbf{i}: u = s(\mathbf{i})\}|, \ u \in I,$$

that is, the count of the number of times the indexing string u occurs as a subsequence in the string s. The associated kernel is defined by

$$\kappa(s,t) = \langle \phi(s), \phi(t) \rangle = \sum_{u\in\Sigma^*} \phi_u(s)\,\phi_u(t).$$

■

An explicit computation of this feature map will be computationally infeasible even if we represent $\phi_u(s)$ by a list of its non-zero components, since if $|s| = m$ we can expect of the order of

$$\min\left(\binom{m}{k}, |\Sigma|^k\right)$$

distinct subsequences of length k. Hence, for $m > 2|\Sigma|$ the number of non-zero entries considering only strings u of length $m/2$ is likely to be at least $|\Sigma|^{m/2}$, which is exponential in the length m of the string s.

Remark 11.15 [Set intersection] We could also consider this as a case of a kernel between multisets, as outlined in Chapter 9. The measure of the intersection is a valid kernel. Neither definition suggests how the computation of the kernel might be effected efficiently. ■

Example 11.16 All the (non-contiguous) subsequences in the words "bar", "baa", "car" and "cat" are given in the following two tables:

ϕ	ε	a	b	c	r	t	aa	ar	at	ba	br	bt
bar	1	1	1	0	1	0	0	1	0	1	1	0
baa	1	2	1	0	0	0	1	0	0	2	0	0
car	1	1	0	1	1	0	0	1	0	0	0	0
cat	1	1	0	1	0	1	0	0	1	0	0	0

ϕ	ca	cr	ct	bar	baa	car	cat
bar	0	0	0	1	0	0	0
baa	0	0	0	0	1	0	0
car	1	1	0	0	0	1	0
cat	1	0	1	0	0	0	1

and since all other (infinite) coordinates must have value zero, the kernel matrix is

K	bar	baa	car	cat
bar	8	6	4	2
baa	6	12	3	3
car	4	3	8	4
cat	2	3	4	8

Notice that in general, the diagonal elements of this kernel matrix can vary greatly with the length of a string and even as illustrated here between words of the same length, when they have different numbers of repeated subsequences. Of course this effect can be removed if we choose to normalise this kernel. Notice also that all the entries are always positive placing all the feature vectors in the positive orthant. This suggests that centering may be advisable in some situations (see Code Fragment 5.2). ■

Evaluating the kernel We now consider how the kernel κ can be evaluated more efficiently than via an explicit computation of the feature vectors. This will involve the definition of a recursive relation similar to that derived for ANOVA kernels in Chapter 9. Its computation will also follow similar dynamic programming techniques in order to complete a table of values with

the resulting kernel evaluation given by the final entry in the table. In presenting this introductory kernel, we will introduce terminology, definitions and techniques that will be used throughout the rest of the chapter.

First consider restricting our attention to the feature indexed by a string u. The contribution of this feature to the overall inner product can be expressed as

$$\phi_u(s)\,\phi_u(t) = \sum_{\mathbf{i}:u=s(\mathbf{i})} 1 \sum_{\mathbf{j}:u=t(\mathbf{j})} 1 = \sum_{(\mathbf{i},\mathbf{j}):u=s(\mathbf{i})=t(\mathbf{j})} 1,$$

where it is understood that $\mathbf{i}$ and $\mathbf{j}$ range over strictly ordered tuples of indices. Hence, the overall inner product can be expressed as

$$\kappa(s,t) = \langle \phi(s), \phi(t) \rangle = \sum_{u \in I} \sum_{(\mathbf{i},\mathbf{j}):u=s(\mathbf{i})=t(\mathbf{j})} 1 = \sum_{(\mathbf{i},\mathbf{j}):s(\mathbf{i})=t(\mathbf{j})} 1. \tag{11.1}$$

The key to the recursion is to consider the addition of one extra symbol a to one of the strings s. In the final sum of equation (11.1) there are two possibilities for the tuple $\mathbf{i}$: either it is completely contained within s or its final index corresponds to the final a. In the first case the subsequence is a subsequence of s, while in the second its final symbol is a. Hence, we can split this sum into two parts

$$\sum_{(\mathbf{i},\mathbf{j}):sa(\mathbf{i})=t(\mathbf{j})} 1 = \sum_{(\mathbf{i},\mathbf{j}):s(\mathbf{i})=t(\mathbf{j})} 1 + \sum_{u:t=uav} \sum_{(\mathbf{i},\mathbf{j}):s(\mathbf{i})=u(\mathbf{j})} 1, \tag{11.2}$$

where for the second contribution we have used the fact that the last character of the subsequence is a which must therefore occur in some position within t.

Computation 11.17 [All-subsequences kernel] Equation (11.2) leads to the following recursive definition of the kernel

$$\begin{aligned} \kappa(s,\varepsilon) &= 1, \\ \kappa(sa,t) &= \kappa(s,t) + \sum_{k:t_k=a} \kappa(s, t(1:k-1)) \end{aligned} \tag{11.3}$$

where we have used the fact that every string contains the empty string ε exactly once. By the symmetry of kernels, an analogous recurrence relation can be given for $\kappa(s,ta)$

$$\begin{aligned} \kappa(\varepsilon,t) &= 1 \\ \kappa(s,ta) &= \kappa(s,t) + \sum_{k:s_k=a} \kappa(s(1:k-1),t). \end{aligned}$$

Similar symmetric definitions will exist for other recurrences given in this chapter, though in future we will avoid pointing out these alternatives. ■

An intuitive reading of the recurrence states that common subsequences are either contained completely within s and hence included in the first term of the recurrence or must end in the symbol a, in which case their final symbol can be matched with occurrences of a in the second string.

Example 11.18 Consider computing the kernel between the strings $s =$ "gatt" and $t =$"cata", where we add the symbol $a =$"a" to the string "gatt" to obtain the string $sa =$ "gatta". The rows of the tables are indexed by the pairs $(\mathbf{i}, \mathbf{j})$ such that $sa(\mathbf{i}) = t(\mathbf{j})$ with those not involving the final character of sa listed in the first table, while those involving the final character are given in the second table. The binary vectors below each string show the positions of the entries in the vectors $\mathbf{i}$ and $\mathbf{j}$.

g	a	t	t	a	$sa(\mathbf{i}) = t(\mathbf{j})$	c	a	t	a
0	0	0	0	0	ε	0	0	0	0
0	1	1	0	0	at	0	1	1	0
0	1	0	1	0	at	0	1	1	0
0	1	0	0	0	a	0	1	0	0
0	0	1	0	0	t	0	0	1	0
0	0	0	1	0	t	0	0	1	0
0	1	0	0	0	a	0	0	0	1

g	a	t	t	a	$sa(\mathbf{i}) = t(\mathbf{j})$	c	a	t	a
0	0	0	0	1	a	0	1	0	0
0	0	0	0	1	a	0	0	0	1
0	1	0	0	1	aa	0	1	0	1
0	0	0	1	1	ta	0	0	1	1
0	0	1	0	1	ta	0	0	1	1
0	1	1	0	1	ata	0	1	1	1
0	1	0	1	1	ata	0	1	1	1

Hence, we see that the 14 rows implying $\kappa(sa, t) = 14$ are made up of the 7 rows of the first table giving $\kappa(s, t) = 7$ plus $\kappa($"gatt", "c"$) = 1$ given by the first row of the second table and $\kappa($"gatt", "cat"$) = 6$ corresponding to rows 2 to 7 of the second table. ■

Cost of the computation As for the case of the ANOVA kernels discussed in Chapter 9 a recursive evaluation of this kernel would be very expensive.

Clearly, a similar strategy of computing the entries into a table based on dynamic programming methods will result in an efficient computation. The table has one row for each symbol of the string s and one column for each symbol of the string t; the entries $\kappa_{ij} = \kappa(s(1:i), t(1:j))$ give the kernel evaluations on the various prefices of the two strings:

DP	ε	t_1	t_2	$\cdots$	t_m
ε	1	1	1	$\cdots$	1
s_1	1	κ_{11}	κ_{12}	$\cdots$	κ_{1m}
s_2	1	κ_{21}	κ_{22}	$\cdots$	κ_{2m}
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$
s_n	1	κ_{n1}	κ_{n2}	$\cdots$	$\kappa_{nm} = \kappa(s,t)$

The first row and column are given by the base case of the recurrence. The general recurrence of (11.3) allows us to compute the (i,j)th entry as the sum of the $(i-1,j)$th entry together with all the entries $(i-1,k-1)$ with $1 \leq k < j$ for which $t_k = s_i$. Clearly, filling the rows in sequential order ensures that at each stage the values required to compute the next entry have already been computed at an earlier stage.

The number of operations to evaluate the single (i,j)th entry is $O(j)$ since we must check through all the symbols in t up to jth. Hence, to fill the complete table will require $O\left(|s|\,|t|^2\right)$ operations using this still slightly naive approach.

Example 11.19 Example 11.18 leads to the following table:

DP	ε	g	a	t	t	a
ε	1	1	1	1	1	1
c	1	1	1	1	1	1
a	1	1	2	2	2	3
t	1	1	2	4	6	7
a	1	1	3	5	7	14

■

Speeding things up We can improve the complexity still further by observing that as we fill the ith row the sum

$$\sum_{k \leq j : t_k = s_i} \kappa(s(1:i-1), t(1:k-1))$$

required when we reach the jth position could have been precomputed into an array P by the following computation

last $= 0$;
$P(0) = 0$;
for $k = 1 : m$
 $P(k) = P(\text{last})$;
 if $t_k = s_i$ then
  $P(k) = P(\text{last}) + \text{DP}(i-1, k-1)$;
  last $= k$;
 end
end

Using the array p we can now compute the next row with the simple loop

for $k = 1 : m$
 $\text{DP}(i,k) = \text{DP}(i-1,k) + P(k)$;
end

We now summarise the algorithm.

Algorithm 11.20 [All-non-contiguous subsequences kernel] The all-non-contiguous subsequences kernel is computed in Code Fragment 11.1. ■

Input	strings s and t of lengths n and m
Process	for $j = 1 : m$
2	$\text{DP}(0,j) = 1$;
3	end
4	for $i = 1 : n$
5	last $= 0$; $P(0) = 0$;
6	for $k = 1 : m$
7	$P(k) = P(\text{last})$;
8	if $t_k = s_i$ then
9	$P(k) = P(\text{last}) + \text{DP}(i-1, k-1)$
10	last $= k$;
11	end
12	end
13	for $k = 1 : m$
14	$\text{DP}(i,k) = \text{DP}(i-1,k) + P(k)$;
15	end
16	end
Output	kernel evaluation $\kappa(s,t) = \text{DP}(n,m)$

Code Fragment 11.1. Pseudocode for the all-non-contiguous subsequences kernel.

Example 11.21 The array p for each row is interleaved with the evaluations of DP for the Example 11.18 in the following table:

DP	ε	g	a	t	t	a
ε	1	1	1	1	1	1
p	0	0	0	0	0	0
c	1	1	1	1	1	1
p	0	0	1	1	1	2
a	1	1	2	2	2	3
p	0	0	0	2	4	4
t	1	1	2	4	6	7
p	0	0	1	1	1	7
a	1	1	3	5	7	14

■

Cost of the computation Since the complexity of both loops individually is $O(|t|)$ their complexity performed sequentially is also $O(|t|)$, making the overall complexity $O(|s||t|)$.

Algorithm 11.20 is deceptively simple. Despite this simplicity it is giving the exponential improvement in complexity that we first observed in the ANOVA computation. We are evaluating an inner product in a space whose dimension is exponential in $|s|$ and in $|t|$ with just $O(|s||t|)$ operations.

Remark 11.22 [Computational by-products] We compute the kernel $\kappa(s,t)$ by solving the *more general* problem of computing $\kappa(s(1:i), t(1:j))$ for all values of $i \leq |s|$ and $j \leq |t|$. This means that at the end of the computation we could also read off from the table the evaluation of the kernel between any prefix of s and any prefix of t. Hence, from the table of Example 11.19 we can see that $\kappa(\texttt{"gatt"}, \texttt{"cat"}) = 6$ by reading off the value of DP$(4,3)$.

■

11.4 Fixed length subsequences kernels

We can adapt the recursive relation presented above in a number of ways, until we are satisfied that the feature space implicitly defined by the kernel is tuned to the particular task at hand.

Our first adaptation reduces the dimensionality of the feature space by only considering non-contiguous substrings that have a given *fixed* length p.

Definition 11.23 [Fixed Length Subsequences Kernel] The feature space associated with the fixed length subsequences kernel of length p is indexed

by Σ^p, with the embedding given by

$$\phi_u^p(s) = |\{\mathbf{i}: u = s(\mathbf{i})\}|, \ u \in \Sigma^p.$$

The associated kernel is defined as

$$\kappa_p(s,t) = \langle \phi^p(s), \phi^p(t) \rangle = \sum_{u \in \Sigma^p} \phi_u^p(s) \phi_u^p(t).$$

■

Evaluating the kernel We can perform a similar derivation as for the all-subsequences kernel except that we must now restrict the index sequences to the set I_p of those with length p

$$\kappa(s,t) = \langle \phi(s), \phi(t) \rangle = \sum_{u \in \Sigma^p} \sum_{(\mathbf{i},\mathbf{j}): u = s(\mathbf{i}) = t(\mathbf{j})} 1 = \sum_{(\mathbf{i},\mathbf{j}) \in I_p \times I_p : s(\mathbf{i}) = t(\mathbf{j})} 1.$$

Following the derivations of the previous section we arrive at

$$\sum_{(\mathbf{i},\mathbf{j}) \in I_p \times I_p : sa(\mathbf{i}) = t(\mathbf{j})} 1 = \sum_{(\mathbf{i},\mathbf{j}) \in I_p \times I_p : s(\mathbf{i}) = t(\mathbf{j})} 1 + \sum_{u : t = uav} \sum_{(\mathbf{i},\mathbf{j}) \in I_{p-1} \times I_{p-1} : s(\mathbf{i}) = u(\mathbf{j})} 1,$$

where, as before, the lengths of the sequences considered in the two components of the sum differ.

Computation 11.24 [Fixed length subsequence kernel] This leads to the following recursive definition of the fixed length subsequences kernel

$$\begin{aligned} \kappa_0(s,t) &= 1, \\ \kappa_p(s,\varepsilon) &= 0, \text{ for } p > 0, \\ \kappa_p(sa,t) &= \kappa_p(s,t) + \sum_{k : t_k = a} \kappa_{p-1}(s, t(1:k-1)). \end{aligned} \tag{11.4}$$

■

Note that now we not only have a recursion over the prefixes of the strings, but also over the lengths of the subsequences considered. The following algorithm includes this extra recursion to compute the kernel by storing the evaluation of the previous kernel in the table DPrec.

Algorithm 11.25 [Fixed length subsequences kernel] The fixed length subsequences kernel is computed in Code Fragment 11.2. ■

Input	strings s and t of lengths n and m length p
Process	$\mathrm{DP}(0:n, 0:m) = 1;$
2	for $l = 1:p$
3	DPrec = DP;
4	for $j = 1:m$
5	$\mathrm{DP}(0,j) = 1;$
6	end
7	for $i = 1:n-p+l$
8	last $= 0$; $P(0) = 0;$
9	for $k = 1:m$
10	$P(k) = P(\text{last});$
11	if $t_k = s_i$ then
12	$P(k) = P(\text{last}) + \mathrm{DPrec}(i-1, k-1);$
13	last $= k$;
14	end
15	end
16	for $k = 1:m$
17	$\mathrm{DP}(i,k) = \mathrm{DP}(i-1,k) + P(k);$
18	end
19	end
20	end
Output	kernel evaluation $\kappa_p(s,t) = \mathrm{DP}(n,m)$

Code Fragment 11.2. Pseudocode for the fixed length subsequences kernel.

Cost of the computation At the pth stage we must be able to access the row above both in the current table and in the table DPrec from the previous stage. Hence, we must create a series of tables by adding an extra loop to the overall algorithm making the overall complexity $O(p|s||t|)$. On the penultimate stage when $l = p - 1$ we need only compute up to row $n - 1$, since this is all that is required in the final loop with $l = p$. In the implementation given above we have made use of this fact at each stage, so that for the lth stage we have only computed up to row $n - p + l$, for $l = 1, \ldots, p$.

Example 11.26 Using the strings from Example 11.18 leads to the following computations for $p = 0, 1, 2, 3$ given in Table 11.3. Note that the sum of all four tables is identical to that for the all-subsequences kernel. This is because the two strings do not share any sequences longer than 3 so that summing over lengths $0, 1, 2, 3$ subsumes all the common subsequences. This also suggests that if we compute the p subsequences kernel we can, at almost no extra cost, compute the kernel κ_l for $l \leq p$. Hence, we have the flexibility

DP : κ_0	ε	g	a	t	t	a	DP : κ_1	ε	g	a	t	t	a
ε	1	1	1	1	1	1	ε	0	0	0	0	0	0
c	1	1	1	1	1	1	c	0	0	0	0	0	0
a	1	1	1	1	1	1	a	0	0	1	1	1	2
t	1	1	1	1	1	1	t	0	0	1	2	3	4
a	1	1	1	1	1	1	a	0	0	2	3	4	6
DP : κ_2	ε	g	a	t	t	a	DP : κ_3	ε	g	a	t	t	a
ε	0	0	0	0	0	0	ε	0	0	0	0	0	0
c	0	0	0	0	0	0	c	0	0	0	0	0	0
a	0	0	0	0	0	0	a	0	0	0	0	0	0
t	0	0	0	1	2	2	t	0	0	0	0	0	0
a	0	0	0	1	2	5	a	0	0	0	0	0	2

Table 11.3. *Computations for the fixed length subsequences kernel.*

to define a kernel that combines these different subsequences kernels with different weightings $a_l \geq 0$

$$\kappa(s,t) = \sum_{l=1}^{p} a_l \kappa_l(s,t).$$

The only change to the above algorithm would be that the upper bound of the loop for the variable i would need to become n rather than $n-p+l$ and we would need to accumulate the sum at the end of the l loop with the assignment

$$\text{Kern} = \text{Kern} + a(l)\,\text{DP}(n,m),$$

where the variable Kern is initialised to 0 at the very beginning of the algorithm. ■

11.5 Gap-weighted subsequences kernels

We now move to a more general type of kernel still based on subsequences, but one that weights the occurrences of subsequences according to how spread out they are. In other words the kernel considers the degree of presence of a subsequence to be a function of how many gaps are interspersed within it. The computation of this kernel will follow the approach developed in the previous section for the fixed length subsequences kernel. Indeed we also restrict consideration to fixed length subsequences in this section.

The kernel described here has often been referred to as the *string kernel* in

ıe literature, though this name has also been used to refer to the p-spectrum ernel. We have chosen to use long-winded descriptive names for the kernels ıtroduced here in order to avoid further proliferating the existing confusion round different names. We prefer to regard 'string kernel' as a generic term pplicable to all of the kernels covered in this chapter.

The key idea behind the gap-weighted subsequences kernel is still to com- are strings by means of the subsequences they contain – the more subse- uences in common, the more similar they are – but rather than weighting ll occurrences equally, the degree of contiguity of the subsequence in the ıput string s determines how much it will contribute to the comparison.

Example 11.27 For example: the string `"gon"` occurs as a subsequence of ıe strings `"gone"`, `"going"` and `"galleon"`, but we consider the first oc- urrence as more important since it is contiguous, while the final occurrence ; the weakest of all three. ■

The feature space has the same coordinates as for the fixed subsequences ernel and hence the same dimension. In order to deal with non-contiguous ıbstrings, it is necessary to introduce a decay factor $\lambda \in (0,1)$ that can be sed to weight the presence of a certain feature in a string. Recall that for n index sequence $\mathbf{i}$ identifying the occurrence of a subsequence $u = s(\mathbf{i})$ ı a string s, we use $l(\mathbf{i})$ to denote the length of the string in s. In the ap-weighted kernel, we weight the occurrence of u with the exponentially ecaying weight $\lambda^{l(\mathbf{i})}$.

Definition 11.28 [Gap-weighted subsequences kernel] The feature space ssociated with the gap-weighted subsequences kernel of length p is indexed y $I = \Sigma^p$, with the embedding given by

$$\phi_u^p(s) = \sum_{\mathbf{i}:u=s(\mathbf{i})} \lambda^{l(\mathbf{i})},\ u \in \Sigma^p.$$

'he associated kernel is defined as

$$\kappa_p(s,t) = \langle \phi^p(s), \phi^p(t) \rangle = \sum_{u \in \Sigma^p} \phi_u^p(s)\, \phi_u^p(t).$$

■

Remark 11.29 [Two limits] Observe that we can recover the fixed length ubsequences kernel by choosing $\lambda = 1$, since the weight of all occurrences

will then be 1 so that

$$\phi_u^p(s) = \sum_{\mathbf{i}:u=s(\mathbf{i})} 1^{l(\mathbf{i})} = |\{\mathbf{i}: u = s(\mathbf{i})\}|, \ u \in \Sigma^p.$$

On the other hand, as $\lambda \to 0$, the kernel approximates the p-spectrum kernel since the relative weighting of strings longer than p tends to zero. Hence, we can view the gap-weighted kernel as interpolating between these two kernels. ■

Example 11.30 Consider the simple strings "cat", "car", "bat", and "bar". Fixing $p = 2$, the words are mapped as follows:

ϕ	ca	ct	at	ba	bt	cr	ar	br
cat	λ^2	λ^3	λ^2	0	0	0	0	0
car	λ^2	0	0	0	0	λ^3	λ^2	0
bat	0	0	λ^2	λ^2	λ^3	0	0	0
bar	0	0	0	λ^2	0	0	λ^2	λ^3

So the unnormalised kernel between "cat" and "car" is κ("cat","car") = λ^4, while the normalised version is obtained using

$$\kappa(\text{"cat"}, \text{"cat"}) = \kappa(\text{"car"}, \text{"car"}) = 2\lambda^4 + \lambda^6$$

as $\hat{\kappa}$("cat","car") = $\lambda^4/(2\lambda^4 + \lambda^6) = (2 + \lambda^2)^{-1}$. ■

11.5.1 Naive implementation

The computation of this kernel will involve both techniques developed for the fixed subsequences kernel and for the p-spectrum kernel. When computing the p-spectrum kernel we considered substrings that occurred as a suffix of a prefix of the string. This suggests considering subsequences whose final occurrence is the last character of the string. We introduce this as an auxiliary kernel.

Definition 11.31 [Gap-weighted suffix subsequences kernel] The feature space associated with the gap-weighted subsequences kernel of length p is indexed by $I = \Sigma^p$, with the embedding given by

$$\phi_u^{p,S}(s) = \sum_{\mathbf{i} \in I_p^{|s|}: u=s(\mathbf{i})} \lambda^{l(\mathbf{i})}, \ u \in \Sigma^p,$$

where we have used I_p^k to denote the set of p-tuples of indices $\mathbf{i}$ with $i_p = k$. The associated kernel is defined as

$$\kappa_p^S(s,t) = \left\langle \phi^{p,S}(s), \phi^{p,S}(t) \right\rangle = \sum_{u \in \Sigma^p} \phi_u^{p,S}(s)\, \phi_u^{p,S}(t).$$

■

Example 11.32 Consider again the simple strings "cat", "car", "bat", and "bar". Fixing $p = 2$, the words are mapped for the suffix version as follows:

$\phi_u^{2,S}(s)$	ca	ct	at	ba	bt	cr	ar	br
cat	0	λ^3	λ^2	0	0	0	0	0
car	0	0	0	0	0	λ^3	λ^2	0
bat	0	0	λ^2	0	λ^3	0	0	0
bar	0	0	0	0	0	0	λ^2	λ^3

Hence, the suffix kernel between "cat" and "car" is κ_2^S("cat","car") $= 0$, while suffix kernel between "cat" and "bat" is κ_2^S("cat","bat") $= \lambda^4$. ■

Figure 11.1 shows a visualisation of a set of strings derived from an embedding given by the gap-weighted subsequences kernel of length 4, with $\lambda = 0.8$. As in the case of the p-spectrum kernel, we can express the gap-

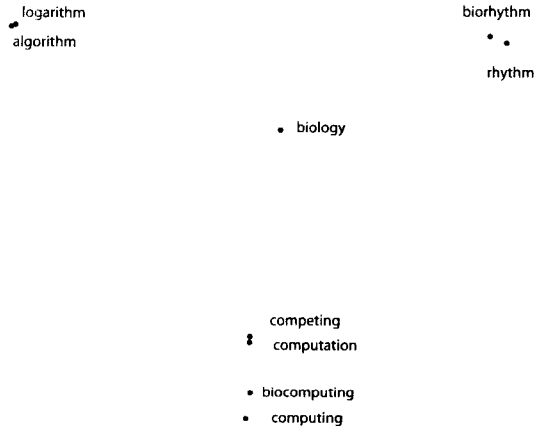

Fig. 11.1. Visualisation of the embedding created by the gap-weighted subsequences kernel.